



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 2
«Реализация метода Гаусса с перестановками»
по курсу «Численные методы линейной алгебры»

Студент группы ИУ9-71Б Лысенко В. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Цель работы

Реализовать три варианта метода Гаусса с перестановками и научиться оценивать погрешность решения системы линейных уравнений для матриц произвольной размерности.

2 Задачи

1. Реализовать метод Гаусса с перестановками по столбцам, по строкам, по столбцам и строкам одновременно для действительных квадратных матриц произвольной размерности n .
2. Для проверки работоспособности алгоритмов необходимо использовать алгоритм тестирования, который заключался в том, что мы заведомо определяем значения координат вектора x , данный вектор является решением уравнения $A \cdot x = b$; вычисляем b путем прямого перемножения матрицы A на вектор x и далее производим поиск решения уравнения $A \cdot x = b$ тем или иным методом Гаусса, получая $x(\text{числ})$, после чего производим сравнение полученного $x(\text{числ})$ с заданным x , а также решением $x(\text{библ})$, полученным с использованием сторонней библиотеки выбранной студеном. При этом сравнение производится по Евклидовой норме разности вектора $x - x(\text{числ})$ и $x - x(\text{библ})$.
3. На защите лабораторной работы студент должен показать умение оценивать погрешность вычислений в зависимости от выполнения условия диагонального преобладания матрицы, умение сравнивать погрешности вычислений полученных методом Гаусса с перестановками по столбцам, по строкам, по столбцам и строкам одновременно. Понимать связь теории с практикой.
4. Результат работы должен быть представлен в виде графиков зависимости абсолютной погрешности вычислений классическим методом Гаусса, методом Гаусса с перестановками по строкам, методом Гаусса с перестановками по столбцам, методом Гаусса с перестановками по столбцам и строкам, библиотечным методом от степени диагонального преобладания.

Все графики должны быть построены на одной координатной плоскости. Напомним, что погрешность вычисления вектора x системы линейных алгебраических уравнений $A \cdot x = b$ тем или иным способом рассчитывается по Евклидовой норме разности точного решения и решения полученного соответствующим методом. Степень диагонального преобладания вычисляется, как максимальная разность по i между модулем диагонального элемента и суммы модулей вне диагональных элементов. Очевидно, что если значение степени диагонального преобладания положительна, то условие диагонального преобладания выполняется, в противном случае — не выполняется. Поэтому график должен быть построен как для отрицательных значений степени диагонального преобладания, так и для положительных.

3 Практическая реализация

В отчёте представлен вариант для квадратной матрицы размером 130x130 и амплитудой значений [-1000:1000]. В качестве заведомого известного вектора x был взят единичный вектор и посчитан вектор F . На основе матрицы A и вектора F с помощью различных методов был вычислен вектор X . В результате посчитаны погрешности вычислений для классического метода Гаусса, метода строк, метода столбцов и комбинированного метода с различными коэффициентами диагонального преобладания.

Ниже представлен код программы:

```
using Plots

n = 130
cup = 1000

function gauss_classic(A, F)
    for k in 1:n-1
        for i in k+1:n
            dif = A[i,k]
            #A[i, :] ./= dif
            F[i] /= dif

            #A[i, :] .*= A[k, k]
            F[i] *= A[k, k]

            #A[i, :] -= A[k, :]
            F[i] -= F[k]
            A[i, :] = A[i, :] ./ dif .* A[k, k] .- A[k, :]
            #F[i:n] = F[i:n] ./ dif .* A[k, k] .- F[k]
        end
    end

    X = ones(Float64, n)
    X[n] = F[n] / A[n,n]
    for i in n-1:-1:1
        dif = 0.0
        for j in i+1:n
            dif += A[i, j] * X[j]
        end
        X[i] = (F[i] - dif) / A[i, i]
    end
    return X
end
```

```

##
function gauss_strings(A, F)
    for k in 1:n-1
        max_i = k
        max_val = 0.0
        for i in k:n
            if (abs(max_val)) < abs(A[i,k])
                max_val = A[i,k]
                max_i = i
            end
        end

        A[k, :] , A[max_i, :] = A[max_i, :] , A[k, :]
        F[k] , F[max_i] = F[max_i] , F[k]

        for i in k+1:n
            dif = A[i,k]
            #A[i, :] ./= dif
            F[i] /= dif

            #A[i, :] .*= A[k, k]
            F[i] *= A[k, k]

            #A[i, :] -= A[k, :]
            F[i] -= F[k]
            A[i, :] = A[i,:] ./ dif .* A[k, k] .- A[k, :]
            #F[i:n] = F[i:n] ./ dif .* A[k, k] .- F[k]
        end
    end

    X = ones(Float64, n)
    X[n] = F[n] / A[n,n]
    for i in n-1:-1:1
        dif = 0.0
        for j in i+1:n
            dif += A[i,j] * X[j]
        end
        X[i] = (F[i] - dif) / A[i, i]
    end
    return X
end
end

```

```

#
function gauss_cols(A, F)

    for k in 1:n-1
        max_i = k
        max_val = 0.0
        for i in k:n
            if (abs(max_val)) < abs(A[k,i])
                max_val = A[k,i]
                max_i = i
            end
        end
        end

        A[:, k], A[:, max_i] = A[:, max_i], A[:, k]

        for i in k+1:n
            dif = A[i,k]
            #A[i, :] ./= dif
            F[i] /= dif

            #A[i, :] .*= A[k, k]
            F[i] *= A[k, k]

            #A[i, :] -= A[k, :]
            F[i] -= F[k]
            A[i, :] = A[i, :] ./ dif .* A[k, k] .- A[k, :]
            #F[i:n] = F[i:n] ./ dif .* A[k, k] .- F[k]
        end
    end

    X = ones(Float64, n)
    X[n] = F[n] / A[n,n]
    for i in n-1:-1:1
        dif = 0.0
        for j in i+1:n
            dif += A[i,j] * X[j]
        end
        X[i] = (F[i] - dif) / A[i, i]
    end
    return X
end

##
function gauss_sting_and_cols(A, F)

```

```

for k in 1:n-1
    max_i = k
    max_val = 0.0
    for i in k:n
        if (abs(max_val)) < abs(A[i,k])
            max_val = A[i,k]
            max_i = i
        end
    end
    end

    A[k, :], A[max_i, :] = A[max_i, :], A[k, :]
    F[k], F[max_i] = F[max_i], F[k]

    max_i = k
    max_val = 0.0
    for i in k:n
        if (abs(max_val)) < abs(A[k,i])
            max_val = A[k,i]
            max_i = i
        end
    end
    end

    A[:, k], A[:, max_i] = A[:, max_i], A[:, k]

    for i in k+1:n
        dif = A[i,k]
        #A[i, :] ./= dif
        F[i] /= dif

        #A[i, :] .*= A[k, k]
        F[i] *= A[k, k]

        #A[i, :] -= A[k, :]
        F[i] -= F[k]
        A[i, :] = A[i, :] ./ dif .* A[k, k] .- A[k, :]
        #F[i:n] = F[i:n] ./ dif .* A[k, k] .- F[k]
    end
end

end
X = ones(Float64, n)
X[n] = F[n] / A[n,n]
for i in n-1:-1:1
    dif = 0.0
    for j in i+1:n
        dif += A[i,j] * X[j]
    end
end

```

```

        X[i] = (F[i] - dif) / A[i, i]
    end
    return X
end

function norm(x)
    sum = 0.0
    for element in x
        sum += element^2
    end
    return sqrt(sum)
end

X = ones(Float64, n)
#X .= 10
A = rand(-cup:cup, n, n)
A = Float64.(A)

F = [Float64(x) for x in (A * X)]

X0 = gauss_classic(copy(A), copy(F))
X1 = gauss_strings(copy(A), copy(F))
X2 = gauss_cols(copy(A), copy(F))
X3 = gauss_sting_and_cols(copy(A), copy(F))

println("Error in the library Gaussian method: ", norm(X - X0))
println("Error in the classical Gaussian method: ", norm(X - X0))
println("Error in the string method: ", norm(X - X1))
println("Error in the column method: ", norm(X - X2))
println("Error in the combined method: ", norm(X - X3))

gauss_classic_array = Vector{Float64}()
gauss_strings_array = Vector{Float64}()
gauss_cols_array = Vector{Float64}()
gauss_sting_and_cols_array = Vector{Float64}()
diag_coefs = Vector{Float64}()

diagonal_increase_coef = -15.0
while diagonal_increase_coef < 15.0
    push!(diag_coefs, diagonal_increase_coef)
    cool_A = copy(A[1:n, 1:n])

```

```

for i in 1:n
    if cool_A[i, i] == 0.0
        cool_A[i, i] += 1
    end
    cool_A[i, i] += 2^diagonal_increase_coef * sign(cool_A[i, i] )
end

F1 = [Float64(x) for x in (cool_A * X)]

push!(gauss_classic_array, norm(X - gauss_classic(copy(cool_A), copy(F1))
))
push!(gauss_strings_array, norm(X - gauss_strings(copy(cool_A), copy(F1))
))
push!(gauss_cols_array, norm(X - gauss_cols(copy(cool_A), copy(F1))))
push!(gauss_sting_and_cols_array, norm(X - gauss_sting_and_cols(copy(
    cool_A), copy(F1))))
diagonal_increase_coef += 1

end

plot(diag_coefs, gauss_classic_array, label="Classic", xlabel="Diagonal coef"
, ylabel="Error")
plot!(diag_coefs, gauss_strings_array, label="Strings", color=:red)
plot!(diag_coefs, gauss_cols_array, label="Columns", color=:yellow)
plot!(diag_coefs, gauss_sting_and_cols_array, label="Combination", color=:
green)

```

4 Результаты

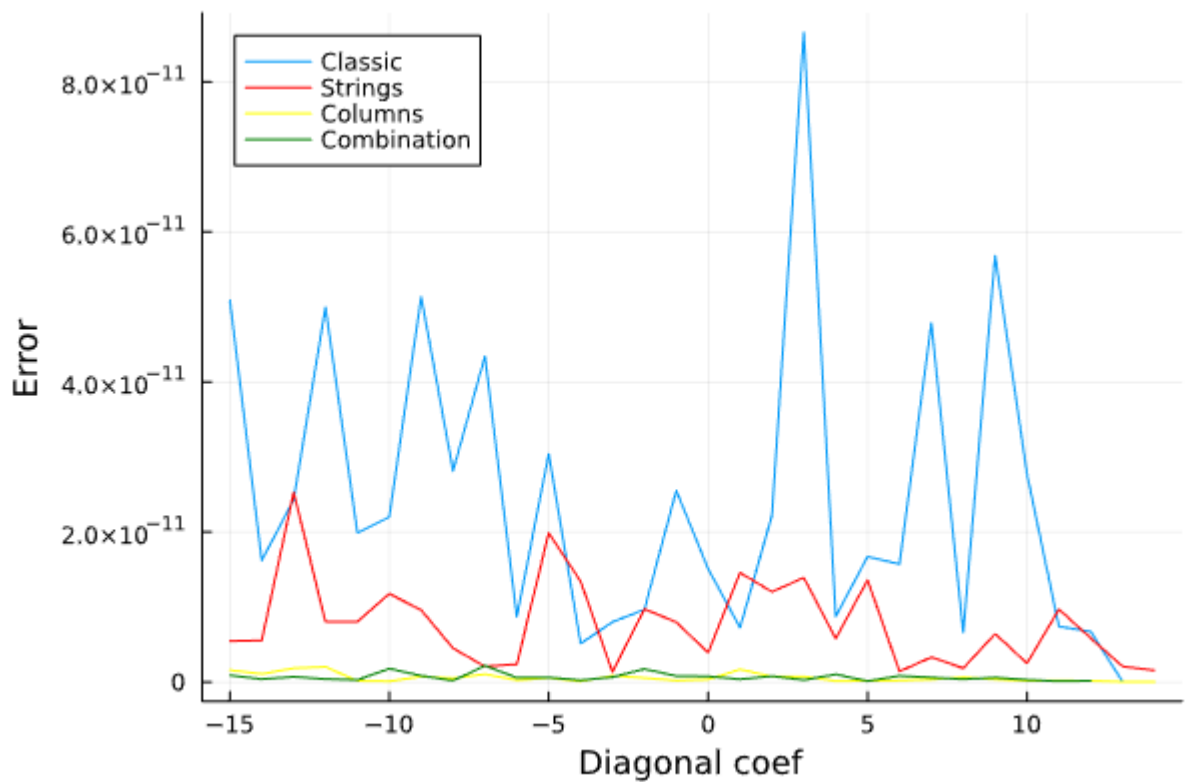
Результат для матрицы с нулевым коэффициентом диагонального преобладания:

```

Error in the library Gaussian method: 2.1855588329906648e-13
Error in the classical Gaussian method: 1.3900766668529158e-11
Error in the string method: 9.007123471212645e-12
Error in the column method: 1.3925100004575132e-12
Error in the combined method: 1.1340334757846268e-12

```

Результаты вычислений различных методов с наращиванием коэффициента диагонального преобладания:



5 Выводы

В ходе выполнения лабораторной работы были реализованы три варианта метода Гаусса с перестановками и посчитаны погрешности решения системы линейных уравнений для матриц произвольной размерности. В сравнении с остальными методами классический метод Гаусса оказался гораздо менее точным. Наиболее надёжным же оказался комбинированный метод.